Now you have a working thread pool that's good for many potential uses. Of course, there are still a myriad of ways to improve it for any particular usage, but that's left as an exercise for the reader. One aspect in particular that hasn't been explored is the idea of dynamically resizing the thread pool to ensure that there's optimal CPU usage even when threads are blocked waiting for something such as I/O or a mutex lock.

Next on the list of "advanced" thread-management techniques is interrupting threads.

9.2. Interrupting threads

In many situations it's desirable to signal to a long-running thread that it's time to stop. This might be because it's a worker thread for a thread pool and the pool is now being destroyed, or it might be because the work being done by the thread has been explicitly canceled by the user, or a myriad of other reasons. Whatever the reason, the idea is the same: you need to signal from one thread that another should stop before it reaches the natural end of its processing, and you need to do this in a way that allows that thread to terminate nicely rather than abruptly pulling the rug from under it.

You could potentially design a separate mechanism for every case where you need to do this, but that would be overkill. Not only does a common mechanism make it easier to write the code on subsequent occasions, but it can allow you to write code that can be interrupted, without having to worry about where that code is being used. The C++11 Standard doesn't provide such a mechanism, but it's relatively straightforward to build one. Let's look at how you can do that, starting from the point of view of the interface for launching and interrupting a thread rather than that of the thread being interrupted.

9.2.1. Launching and interrupting another thread

To start with, let's look at the external interface. What do you need from an interruptible thread? At the basic level, all you need is the same interface as